

Основы работы с DOM в JavaScript

В предыдущих уроках мы изучали базовые возможности языка JavaScript. Начиная с данного урока мы займемся тем, для чего собственно JavaScript и предназначен - мы будем изменять элементы HTML страницы и реагировать на действия пользователя. Наши скрипты станут более зрелищными и полезными.

Во всех следующих уроках ваш JavaScript код должен быть размещен ниже HTML кода, к которому он обращается. То есть структура вашего кода должна выглядеть следующим образом:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4
5    </head>
6    <body>
7      ваши html теги
8
9    <script>
10     ваш JavaScript код
11   </script>
12 </body>
13 </html>
```

Либо следующим образом, если вы пишете JavaScript код в отдельном файле:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4
5    </head>
6    <body>
7      ваши html теги
8
9    <script src="index.js"></script>
10 </body>
11 </html>
```

В дальнейшем для краткости я не буду приводить полный код, а буду просто указывать, что нужно писать в HTML части, а что - в JavaScript части.

№ 1

⊗jsPmDmInr

Сделайте файл с указанной структурой кода.

DOM элементы в JavaScript

Каждому тегу страницы в JavaScript соответствует некоторый объект со своими свойствами. В этом объекте хранится текст тега, значения его атрибутов, а также другие полезные вещи. В JavaScript такие объекты называют *DOM элементами*.

Пусть в нашем HTML коде есть следующий тег:

```
1 <div id="elem">text</div>
```

Пусть в переменной `elem` лежит ссылка на этот тег. В этом случае переменная `elem` будет объектом со свойствами `id` и `textContent`. Свойство `id` будет содержать значение атрибута `id` нашего тега, а свойство `textContent` - его текст.

В следующем уроке мы научимся получать в JavaScript ссылки на теги страницы.

Получение DOM элемента в JavaScript

Сейчас мы научимся получать DOM элементы страницы, чтобы в дальнейшем производить с ними какие-нибудь манипуляции.

Пусть у нас есть некоторая кнопка:

```
1 <input id="button" type="submit">
```

Давайте получим ссылку на эту кнопку в переменную. Для этого следует использовать метод `querySelector` специального объекта `document`. Этот метод параметром принимает CSS селектор и возвращает ссылку на найденный по этому селектору элемент.

У нашей кнопочки есть атрибут `id` со значением `button`. Значит, мы можем найти это кнопку по селектору `#button`. Итак, давайте найдем нашу кнопочку и запишем ссылку на нее в переменную:

```
1 let button = document.querySelector('#button');  
2 console.log(button);
```

№ 1

⊗ jsPmDmEG

Даны 3 абзаца:

```
1 <p id="elem1">1</p>  
2 <p id="elem2">2</p>  
3 <p id="elem3">3</p>
```

Запишите ссылку на каждый из абзацев в отдельную переменную и выведите содержимое каждой из этих переменных в консоль.

Сложные селекторы DOM элемента в JavaScript

Давайте сделаем что-нибудь более сложное. Пусть, к примеру, у нас есть вот такой HTML код:

```
1 <div id="parent">
2   <input>
3 </div>
```

Давайте получим ссылку на инпут, находящийся внутри блока #parent:

```
1 let elem = document.querySelector('#parent input');
2 console.log(elem);
```

Как вы видите, мы получили наш инпут по селектору #parent input. На самом деле под этот CSS селектор попадают все инпуты, находящиеся в #parent.

Пусть, к примеру, у нас два таких инпута:

```
1 <div id="parent">
2   <input value="1">
3   <input value="2">
4 </div>
```

Давайте получим ссылку на инпут, находящийся внутри блока #parent:

```
1 let elem = document.querySelector('#parent input');
2 console.log(elem);
```

Как вы видите, мы получили наш инпут по селектору #parent input. На самом деле под этот CSS селектор попадают все инпуты, находящиеся в #parent.

Пусть, к примеру, у нас два таких инпута:

```
1 <div id="parent">
2   <input value="1">
3   <input value="2">
4 </div>
```

Тогда, если написать селектор #parent input в CSS коде, он затронет оба наших инпута:

```
1 #parent input {
2   color: red;
3 }
```

Метод `querySelector`, однако, работает не так. Он всегда получает только один элемент - первый, попавший под указанный селектор. Давайте посмотрим на примере. Пусть у нас есть инпуты с классом `elem`:

```
1 <input class="elem">
2 <input class="elem">
```

Давайте получим первый из этих инпутов:

```
1 let elem = document.querySelector('.elem');
2 console.log(elem); // здесь будет первый инпут
```

№ 1

⊗jsPmDmEGAS

Дан следующий HTML:

```
1 <div id="block">
2   <p>1</p>
3   <p>2</p>
4 </div>
```

Получите ссылку на первый абзац из дива с id, равным block.

№ 2

⊗jsPmDmEGAS

Дан следующий HTML:

```
1 <div class="block">
2   <p>1</p>
3   <p>2</p>
4 </div>
```

Получите ссылку на первый абзац из дива с классом block.

№ 3

⊗jsPmDmEGAS

Дан следующий HTML:

```
1 <p class="www">text</p>
2 <p class="www">text</p>
```

Получите ссылку на первый абзац с классом www.

Привязывание обработчиков к элементам в JavaScript

Давайте теперь научим наши DOM элементы реагировать на действия пользователя сайта. Например, пользователь нажмет куда-либо мышкой, а наш код в ответ должен будет обработать это нажатие и вывести на экран какую-либо информацию.

Действия пользователя, которые мы можем отследить через JavaScript, называются *событиями*. События могут быть следующими: клик мышкой на элемент страницы, наведение мышкой на элемент страницы или наоборот - уход курсора мыши с элемента и так далее. Кроме того, есть события, не зависящие от действий пользователя, например, событие по загрузке HTML страницы в браузер.

Давайте для примера сделаем кнопку, по нажатию на которую на экран выведется какой-то текст. Для начала сделаем HTML код кнопки:

```
1 <input id="button" type="submit">
```

Получим теперь ссылку на кнопку в переменную:

```
1 let button = document.querySelector('#button');
```

Теперь нам необходимо задать реакцию нашей кнопки при клике по ней. Для этого в JavaScript существует специальный метод `addEventListener`, первым параметром принимающий название события (клик на кнопку имеет название `'click'`), а вторым параметром - *функцию-коллбэк*, выполняющуюся при возникновении этого события.

Давайте, например, по клику на кнопку выведем какой-нибудь текст:

```
1 button.addEventListener('click', function() {  
2   console.log('!!!');  
3 });
```

№ 1

©jsPmDmEHB

Даны 3 кнопки:

```
1 <input id="button1" type="submit">  
2 <input id="button2" type="submit">  
3 <input id="button3" type="submit">
```

Сделайте так, чтобы по клику на первую кнопку на экран выводилось число 1, по клику на вторую - число 2, а по клику на третью - число 3.

Именованные обработчики событий в JavaScript

В предыдущих уроках мы использовали в качестве обработчиков событий анонимные функции. Это на самом деле не обязательно - функция может быть и обычной, с именем. Пусть для примера у нас есть такая функция:

```
1 function func() {  
2   console.log('!!!');  
3 }
```

Пусть также есть кнопка:

```
1 <input id="button" type="submit">
```

Давайте сделаем так, чтобы по клику на кнопку выполнялась наша функция `func`. Для этого параметром `addEventListener` передадим имя нашей функции, вот так:

```
1 let button = document.querySelector('#button');  
2 button.addEventListener('click', func);  
3  
4 function func() {  
5   console.log('!!!');  
6 }
```

№ 1

⊗jsPmDmNEH

Дан следующий HTML код:

```
1 <input id="button1" type="submit" value="button1">  
2 <input id="button2" type="submit" value="button2">
```

Даны следующие функции:

```
1 function func1() {  
2   console.log(1);  
3 }  
4  
5 function func2() {  
6   console.log(2);  
7 }
```

Сделайте так, чтобы по клику на первую кнопку выполнялась функция `func1`, а по клику на вторую - функция `func2`.

Один обработчик ко многим элементам в JavaScript

Одну функцию можно привязать сразу к нескольким элементам. Пусть для примера у нас есть следующая функция:

```
1 function func() {  
2   console.log('!!!');  
3 }
```

Есть также две кнопки:

```
1 <input id="button1" type="submit" value="button1">  
2 <input id="button2" type="submit" value="button2">
```

Получим ссылки на эти кнопки в переменные:

```
1 let button1 = document.querySelector('#button1');  
2 let button2 = document.querySelector('#button2');
```

А теперь давайте привяжем нашу функцию func и к первой, и ко второй кнопке:

```
1 button1.addEventListener('click', func);  
2 button2.addEventListener('click', func);
```

№ 1

⊗jsPmDmMEH

Даны 5 абзацев:

```
1 <p id="elem1">text</p>  
2 <p id="elem2">text</p>  
3 <p id="elem3">text</p>  
4 <p id="elem4">text</p>  
5 <p id="elem5">text</p>
```

Дана следующая функция:

```
1 function func() {  
2   console.log('message');  
3 }
```

Привяжите эту функцию ко всем 5-ти абзацам.

Несколько обработчиков одного события в JavaScript

К одному элементу можно привязать сразу несколько функций. Давайте посмотрим на примере. Пусть у нас есть кнопка:

```
1 <input id="button" type="submit">
```

Пусть у нас есть две функции:

```
1 function func1() {  
2   console.log('1');  
3 }  
4  
5 function func2() {  
6   console.log('2');  
7 }
```

Получим ссылку на нашу кнопку в переменную:

```
1 let button = document.querySelector('#button');
```

А теперь давайте привяжем к нашей кнопке в качестве обработчиков клика и первую, и вторую функции:

```
1 button.addEventListener('click', func1);  
2 button.addEventListener('click', func2);
```

№ 1

⊗jsPmDmSH

Дан абзац:

```
1 <p id="elem">text</p>
```

Даны следующие функции:

```
1 function func1() {  
2   console.log('1');  
3 }  
4  
5 function func2() {  
6   console.log('2');  
7 }  
8  
9 function func3() {  
10  console.log('3');  
11 }
```

Привяжите все эти функции к нашему абзацу.

Обработчики разных событий в JavaScript

Кроме клика по элементу, существуют и другие события. Например, с помощью события `dblclick` можно отловить двойной клик по элементу, с помощью события `mouseover` - наведение курсора на элемент, а с помощью события `mouseout` - уход курсора с элемента.

При этом к одному элементу можно привязывать обработчики различных типов событий. Давайте, например, привяжем к одному элементу реакцию на наведение курсора и реакцию на уход:

```
1 button.addEventListener('mouseover', function() {  
2   console.log('1');  
3 });  
4 button.addEventListener('mouseout', function() {  
5   console.log('2');  
6 });
```

№1

⊗jsPmDmDEH

Дана кнопка. По двойному клику по ней выведите какое-нибудь сообщение.

№2

⊗jsPmDmDEH

Дана кнопка. По наведению на нее выведите какое-нибудь сообщение.

№3

⊗jsPmDmDEH

Дана кнопка. По уходу курсора с нее выведите какое-нибудь сообщение.

№4

⊗jsPmDmDEH

Дана кнопка. По наведению на нее выведите одно сообщение, а по уходу с нее - другое.

Работа с текстом элементов на JavaScript

У DOM элементов есть свойство `textContent`, позволяющее прочитывать текст этих элементов. Давайте посмотрим на примере. Пусть у нас есть следующий тег:

```
1 <p id="elem">text</p>
```

Получим ссылку на этот тег в переменную:

```
1 let elem = document.querySelector('#elem');
```

Прочитаем текст тега:

```
1 console.log(elem.textContent);
```

Поменяем текст тега:

```
1 elem.textContent = '!!!!';
```

№1

⊗jsPmDmET

Дан абзац и кнопка. По клику на кнопку прочитайте текст абзаца и выведите его в консоль.

№2

⊗jsPmDmET

Дан абзац и кнопка. По клику на кнопку запишите в абзац новый текст.

№3

⊗jsPmDmET

Даны два абзаца, содержащие своим текстом какие-то числа, и кнопка. По нажатию на кнопку выведите на экран сумму хранящихся чисел.

№4

⊗jsPmDmET

Даны три абзаца с числами, див, и кнопка. По нажатию на кнопку запишите в текст дива сумму чисел.

№5

⊗jsPmDmET

Дан абзац с числом и кнопка. По нажатию на кнопку прибавьте к значению абзаца единицу и запишите полученное число обратно.

№6

⊗jsPmDmET

Дан абзац с текстом и кнопка. По нажатию на кнопку запишите в конец текста абзаца восклицательный знак.

Работа с HTML кодом элементов на JavaScript

У DOM элементов есть свойство `innerHTML`, позволяющее прочитывать HTML код этих элементов. Давайте посмотрим на примере. Пусть у нас есть следующий тег:

```
1 | <p id="elem"><b>text</b></p>
```

Получим ссылку на этот тег в переменную:

```
1 | let elem = document.querySelector('#elem');
```

Прочитаем HTML код тега:

```
1 | console.log(elem.innerHTML); // выведет <b>text</b>
```

Поменяем текст тега:

```
1 | elem.innerHTML = '<i>!!!</i>';
```

№1

⊗jsPmDmEH

Дан абзац и кнопка. По клику на кнопку прочитайте HTML код абзаца и выведите его в консоль.

№2

⊗jsPmDmEH

Дан абзац и кнопка. По клику на кнопку запишите в абзац новый текст так, чтобы он был жирным.

Атрибуты тегов как свойства в JavaScript

Давайте теперь научимся получать атрибуты тегов. Здесь действует следующее правило: *каждому атрибуту тега соответствует одноименное свойство DOM элемента.*

Давайте посмотрим на примере. Пусть у нас есть вот такой тег:

```
1 | <input id="elem" type="text">
```

Получим ссылку на наш элемент в переменную:

```
1 | let elem = document.querySelector('#elem');
```

Выведем значения нужных нам атрибутов:

```
1 console.log(elem.id); // выведет 'elem'
2 console.log(elem.type); // выведет 'text'
```

А теперь для примера поменяем значение атрибута:

```
1 elem.type = 'submit';
```

№1

⊗ jsPmDmAVP

Дан следующий инпут:

```
1 <input id="elem" type="email">
```

Дана также кнопка. По нажатию на кнопку выведите на экран содержимое атрибута type указанного выше инпута.

№2

⊗ jsPmDmAVP

Дан следующий инпут:

```
1 <input id="elem" type="email">
```

Дана также кнопка. По нажатию на кнопку запишите в атрибут type значение submit.

№3

⊗ jsPmDmAVP

Пусть у вас есть ссылка в виде тега a, кнопка и абзац. По нажатию на кнопку выведите в абзац содержимое атрибута href ссылки.

№4

⊗ jsPmDmAVP

Пусть у вас есть ссылка и кнопка. По нажатию на кнопку добавьте в конец текста ссылки содержимое ее атрибута href в круглых скобках.

№5

⊗jsPmDmAVP

Пусть у вас есть картинка. Разместите ее на странице с помощью тега `img`. Пусть даны также кнопка и абзац. Сделайте так, чтобы по клику на кнопку в абзац записался путь к картинке из ее атрибута `src`.

№6

⊗jsPmDmAVP

Дана картинка в теге `img` и кнопка. По нажатию на кнопку в атрибут `width` запишите значение 300.

№7

⊗jsPmDmAVP

Дана картинка в теге `img` и кнопка. Пусть в атрибуте `width` задана некоторая ширина. Сделайте кнопку, по нажатию на которую ширина картинки будет увеличиваться в 2 раза.

№8

⊗jsPmDmAVP

Пусть у вас есть две картинки. Сделайте на странице тег `img` и две кнопки. По нажатию на первую кнопку запишите в атрибут `src` путь к первой картинке, а по нажатию на вторую - путь ко второй картинке.

Работа с текстовыми полями в JavaScript

Сейчас мы научимся получать текст от пользователей нашего сайта. Для этого в HTML предусмотрен специальный тег `input`, представляющий собой текстовое поле для ввода данных.

```
1 | <input>
```

У данного тега есть специальный атрибут `value`, задающий начальный текст, который будет написан в инпуте по заходу на страницу:

```
1 | <input value="text">
```

Пользователь нашего сайта после захода на страницу может поменять текст инпута. При этом, если у нас есть переменная, содержащая ссылку на этот инпут, то свойство `value` этой переменной всегда будет содержать текущее значение текста инпута.

Попробуем на практике. Пусть у нас инпут с атрибутом `value`:

```
1 | <input id="elem" value="text">
```

Получим ссылку на этот инпут в переменную:

```
1 let elem = document.querySelector('#elem');
```

А теперь выведем на экран текущий текст инпута:

```
1 console.log(elem.value);
```

А теперь поменяем текст инпута на другой:

```
1 elem.value = 'new text';
```

№1

⊗ jsPmDmTF

Дан инпут и кнопка. По нажатию на кнопку запишите в инпут какой-нибудь текст.

№2

⊗ jsPmDmTF

Дан инпут, абзац и кнопка. По нажатию на кнопку запишите в абзац текст из инпута.

№3

⊗ jsPmDmTF

Даны два инпута и кнопка. В первый инпут пользователем вводится число. По нажатию на кнопку запишите во второй инпут квадрат введенного числа.

№4

⊗ jsPmDmTF

Даны два инпута и кнопка. По нажатию на кнопку запишите в первый инпут значение второго инпута, а во второй инпут - значение первого. Ваш код должен работать универсально, для любых значений инпутов.

№5

⊗ jsPmDmTF

Даны 5 инпутов, абзац и кнопка. В инпут вводятся числа. По нажатию на кнопку запишите среднее арифметическое введенных чисел в абзац.

Фокус текстовых полей в JavaScript

Пусть у нас есть инпут. Вы можете нажать на этот инпут и после этого в нем заморгает курсор-палочка и можно будет вводить в него текст.

Про такое состояние говорят, что инпут сейчас имеет *фокус ввода*. На практике это означает, что если начать вводить текст с клавиатуры, то этот текст будет попадать в тот инпут, который сейчас имеет фокус ввода. Если затем кликнуть куда-нибудь вне инпута, то этот инпут *потеряет* фокус ввода и в него нельзя будет вводить текст.

Для того, чтобы поймать момент получения или потери фокуса инпутом, в JavaScript предусмотрены специальные события: событие `focus` позволяет отловить получение фокуса инпутом, а событие `blur` - потерю. Попробуем на практике. Пусть у нас инпут:

```
1 <input id="elem" value="text">
```

Получим ссылку на него в переменную:

```
1 let elem = document.querySelector('#elem');
```

А теперь сделаем так, чтобы по получению фокуса в консоль вывелся текущий текст инпута:

```
1 elem.addEventListener('focus', function() {  
2   console.log(elem.value);  
3 });
```

№1

⊗ jsPmDmTFF

Дан инпут. По получению фокуса запишите в него число 1, а по потере фокуса - число 2.

№2

⊗ jsPmDmTFF

Дан инпут. Пусть в него вводится число. По потере фокуса выведите на экран квадрат этого числа.

№3

⊗ jsPmDmTFF

Дан инпут, в котором изначально есть какой-то текст. По получению фокуса инпутом очистите содержимое этого инпута.

Исключения при работе с атрибутами в JavaScript

При работе с атрибутами существует исключение - это атрибут `class`. Это слово является специальным в JavaScript и поэтому мы не можем просто написать `elem.class`, чтобы считать значение атрибута `class`. Вместо этого следует писать `elem.className`.

Давайте посмотрим на примере. Пусть у нас дан вот такой инпут:

```
1 <input id="elem" class="aaa bbb">
```

Давайте выведем значение атрибута class для нашего инпута:

```
1 let elem = document.querySelector('#elem');  
2 console.log(elem.className); // выведет 'aaa bbb'
```

Существуют и другие исключения, например, атрибут for. К нему следует обращаться через htmlFor.

№1

⊗ jsPmDmEA

Дан див:

```
1 <div id="elem" class="content no-gap"></div>
```

Дана также кнопка. По нажатию на кнопку прочитайте и выведите на экран значение атрибута class нашего дива.

№2

⊗ jsPmDmEA

Дан див:

```
1 <div id="elem"></div>
```

Дана также кнопка. По нажатию на кнопку запишите в атрибут class нашего дива какой-нибудь класс.

№3

⊗ jsPmDmEA

Дан див с несколькими CSS классами, записанными через пробел:

```
1 <div id="elem" class="aaa bbb ccc"></div>
```

Дана также кнопка. По нажатию на кнопку получите массив CSS классов нашего дива.

Цепочки методов и свойств в JavaScript

Пусть у нас есть инпут:

```
1 <input id="elem" value="text">
```


Давайте выведем на экран текст инпута:

```
1 let elem = document.querySelector('#elem');
2 console.log(elem.value); // выведет 'text'
```

Как вы видите, мы сначала получаем элемент по его id, записываем этот элемент в переменную elem, а затем выводим на экран свойство value из этой переменной.

На самом деле можно не вводить переменную elem, а строить цепочку из точек таким образом:

```
1 console.log( document.querySelector('#elem').value ); // выведет 'text'
```

Таким же образом - цепочкой - можно производить и перезапись атрибутов:

```
1 document.querySelector('#elem').value = 'www';
```

№ 1

⊗jsPmDmChn

Дан следующий код:

```
1 

1 let image = document.querySelector('#image');
2 console.log(image.src);
```

Переделайте приведенный выше код так, чтобы вместо введения переменной image использовалась цепочка.

Преимущества и недостатки цепочек в JavaScript

Не смотря на то, что цепочки сокращают код, в большинстве случаев введение переменной все-таки удобнее. Сравните два примера - сейчас я ввел переменную elem и могу записывать любое количество атрибутов, при этом querySelector вызывается только один раз:

```
1 let elem = document.querySelector('#elem');
2 elem.value = 'www';
3 elem.type = 'submit';
```

А сейчас я не ввожу новую переменную и поэтому мне приходится вызывать querySelector два раза:

```
1 document.querySelector('#elem').value = 'www';
2 document.querySelector('#elem').type = 'submit';
```

На мой взгляд, этот код стал сложнее, хотя и занимает на одну строчку меньше. Кроме того, если я захочу сменить значение id с 'elem' на какое-то другое, мне придется делать это во многих местах, что не очень удобно.

Есть и еще проблема - нагрузка на браузер. Поиск элементов по странице, который делает метод querySelector, является довольно медленной операцией (*и вообще любая работа с элементами страницы - это медленная операция - запомните это*).

В нашем случае, если мы каждый раз используем `querySelector`, то браузер каждый раз будет обрабатывать HTML страницу и искать элемент с заданным `id` несколько раз (не важно, что `id` одинаковые - браузер проделает все действия несколько раз), совершая бесполезные операции, которые могут замедлить работу браузера.

Если же мы используем переменную `elem` - никакого поиска по странице не происходит (элемент уже найден и ссылка на него лежит в переменной).

№1

⊗jsPmDmChA

Дан следующий код:

```
1 | 

1 | console.log(document.querySelector('#image').src);
2 | console.log(document.querySelector('#image').width);
3 | console.log(document.querySelector('#image').height);
```

Укажите на недостатки данного кода. Исправьте их.

Объект this в JavaScript

Сейчас мы будем работать со специальным объектом `this`, доступным в функции-обработчике события. Этот объект указывает на элемент, в котором произошло событие.

Объект `this` удобен, когда элемент, в котором произошло событие, и элемент, с которым совершаются действия в результате события, - это один и тот же элемент.

Например, если у нас есть инпут, мы можем привязать к нему обработчик потери фокуса и по наступлению этого события что-то сделать с текстом инпута. Давайте сделаем описанное. Пусть у нас дан инпут:

```
1 | <input id="elem" value="text">
```

Давайте получим ссылку на него в переменную `elem`:

```
1 | let elem = document.querySelector('#elem');
```

Привяжем к нему функцию-обработчик события `blur`:

```
1 | elem.addEventListener('blur', func);
```

Внутри этой функции `func` будет доступен объект `this`, указывающий на наш инпут:

```
1 | function func() {
2 |   console.log(this); // содержит ссылку на наш элемент
3 | }
```

Выведем содержимое атрибута `value` нашего инпута:

```
1 | function func() {
2 |   console.log(this.value); // выведем содержимое атрибута
3 | }
```

Ну, а теперь запишем в инпут какой-нибудь текст:

```
1 function func() {  
2   this.value = '!!!';  
3 }
```

Можно использовать и анонимную функцию:

```
1 elem.addEventListener('blur', function() {  
2   this.value = '!!!';  
3 });
```

№1

⊗jsPmDmThs

Дан инпут. По получению фокуса этим инпутом запишите в него число 1, а по потере фокуса - число 2. Для обращения у инпуту внутри функции-обработчика используйте объект `this`.

№2

⊗jsPmDmThs

Дана кнопка, значением которой служит число 1. Сделайте так, чтобы по клику на эту кнопку ее значение каждый раз увеличивалось на единицу.

Преимущество `this` в JavaScript

Из сказанного ранее пока не очевидно особое преимущество `this`. Ведь внутри функции-обработчика и так будет доступен наш элемент - ведь переменная `elem` будет глобальной для нашей функции `func`:

```
1 let elem = document.querySelector('#elem');  
2 elem.addEventListener('click', func);  
3  
4 function func() {  
5   // здесь доступна переменная elem с нашим элементом  
6 }
```

И, несложно сообразить, что содержимое `this` и содержимое переменной `elem` в нашем случае равны:

```
1 let elem = document.querySelector('#elem');  
2 elem.addEventListener('click', func);  
3  
4 function func() {  
5   console.log(elem === this); // выведет true  
6 }
```

В чем же особое преимущество `this`? Оно проявляется, когда у нас несколько элементов, и к каждому привязана одна и та же функция.

Посмотрим на примере. Пусть у нас есть 3 кнопки:

```
1 <input id="button1" type="submit" value="text1">
2 <input id="button2" type="submit" value="text2">
3 <input id="button3" type="submit" value="text3">
```

Получим ссылки на них в переменные:

```
1 let button1 = document.querySelector('#button1');
2 let button2 = document.querySelector('#button2');
3 let button3 = document.querySelector('#button3');
```

Привяжем к этим кнопкам одну и ту же функцию:

```
1 button1.addEventListener('click', func);
2 button2.addEventListener('click', func);
3 button3.addEventListener('click', func);
```

А внутри функции будем выводить `this.value`:

```
1 function func() {
2   console.log(this.value);
3 }
```

Получится, что у нас есть три кнопки. Нажатие на каждую кнопку будет приводить к вызову функции `func`. При этом при каждом клике `this` будет содержать ссылку на ту кнопку, в которой произошло событие.

То есть каждое нажатие будет выводить в консоль `value` той кнопки, на которой произошло нажатие, но делать это будет одна и та же функция `func`! Вот в чем преимущество использования `this`.

№ 1

©jsPmDmThsA

Даны 5 абзацев с какими-то текстами. По клику на любой абзац запишите в конец его текста восклицательный знак.

№ 2

©jsPmDmThsA

Даны 3 инпута, в которых записаны какие-то числа. По потере фокуса в любом из инпутов возведите стоящее в нем число в квадрат.

Получение группы элементов в JavaScript

В предыдущих уроках мы с помощью метода `querySelector` получали один элемент страницы. Теперь пришло время научиться получать группу элементов и проделывать какие-нибудь операции сразу со многими элементами.

Для этого существует метод `querySelectorAll`, получающий все теги, подпадающие под CSS селектор, в виде массива элементов. Чтобы сделать что-нибудь с найденными элементами, нужно поработать с полученным массивом, например, перебрать его циклом и в цикле выполнить какую-либо операцию с каждым элементом по отдельности.

Пусть, к примеру, у нас даны абзацы с классом `www`:

```
1 <p class="www">text1</p>
2 <p class="www">text2</p>
3 <p class="www">text3</p>
```

Давайте получим массив этих абзацев, переберем их циклом и в цикле выведем тексты найденных абзацев в консоль:

```
1 let elems = document.querySelectorAll('.www');
2
3 for (let elem of elems) {
4   console.log(elem.textContent);
5 }
```

А теперь давайте в конец текста каждого абзаца добавим восклицательный знак:

```
1 let elems = document.querySelectorAll('.www');
2
3 for (let elem of elems) {
4   elem.textContent = elem.textContent + '!';
5 }
```

№ 1

⊗jsPmDmEGG

Даны абзацы и кнопка. По нажатию на кнопку найдите все абзацы, переберите их циклом и установите текст каждого абзаца в значение `'text'`.

№ 2

⊗jsPmDmEGG

Даны абзацы с текстом и кнопка. По нажатию на кнопку запишите в конец текста каждого абзаца его порядковый номер.

№ 3

⊗jsPmDmEGG

Даны инпуты с числами, абзац и кнопка. По нажатию на кнопку найдите сумму чисел из инпутов и запишите эту сумму в текст абзаца.

Добавление обработчиков в цикле в JavaScript

Давайте теперь научимся массово добавлять обработчики событий элементам. Пусть, к примеру, у нас есть абзацы:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Пусть у нас также есть функция:

```
1 function func() {
2   console.log('!');
3 }
```

Давайте переберем наши абзацы в цикле и каждому абзацу добавим в качестве обработчика клика функцию func:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', func);
5 }
```

Давайте пойдем дальше и сделаем так, чтобы по клику на любой абзац выводился текст этого абзаца. Есть, однако, проблема: абзацев много, а функция-обработчик одна. Как же нам отличить наши абзацы внутри функции-обработчика?

В этом нам поможет объект `this` - при вызове функции в момент события этот объект будет указывать на тот элемент, где это событие случилось. Переделаем код нашей функции `func` в соответствии со сказанным:

```
1 function func() {
2   console.log(this.textContent); // выводим текст абзаца
3 }
```

№1

©jsPmDmLNH

Дана следующая функция:

```
1 function func() {
2   this.value = Number(this.value) + 1;
3 }
```

Даны также инпуты. Сделайте так, чтобы по потере фокуса в любом из наших инпутов выполнялась приведенная выше функция.

№2

©jsPmDmLNH

Даны абзацы с числами. Сделайте так, чтобы по клику на любой абзац его число в нем возводилось в квадрат.

Добавление анонимных обработчиков в цикле в JavaScript

В качестве обработчиков, навешиваемых в цикле, можно использовать и анонимные функции. Это делает код более компактным и избавляет нас от придумывания имени для функции, которая используется только в одном месте.

Давайте навесим на элементы анонимные обработчики:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     console.log(this.textContent);
6   });
7 }
```

№1

⊗jsPmDmLАН

Дан следующий код:

```
1 <div>1</div>
2 <div>2</div>
3 <div>3</div>
4 <div>4</div>
5 <div>5</div>
6
7 let divs = document.querySelectorAll('div');
8
9 for (let div of divs) {
10   div.addEventListener('click', func);
11 }
12
13 function func() {
14   this.textContent++;
15 }
```

Сделайте функцию-обработчик анонимной.

Отвязывание событий в JavaScript

В данном уроке мы научимся отвязывать обработчики события, которые ранее были привязаны нами к элементам. Пусть для примера дана следующая кнопка:

```
1 <input id="button" type="submit">
```

Привяжем к этой кнопке функцию func:

```
1 let button = document.querySelector('#button');
2 button.addEventListener('click', func);
3
4 function func() {
5   console.log('!!!');
6 }
```

Давайте теперь сделаем так, чтобы обработчик события срабатывал на первый клик, а потом отвязывался от кнопки. Для этого существует специальный метод `removeEventListener`. Этот метод первым параметром принимает тип события, а вторым - ссылку на функцию, которую нужно отвязать.

Как правило, это значит, что обработчик события отвязывается так же, как и привязывался. То есть, если мы привязали его вот так: `addEventListener('click', func)`, то и отвяжем с теми же параметрами, вот так: `removeEventListener('click', func)`.

Итак, решим поставленную нами задачу:

```
1 let button = document.querySelector('#button');
2 button.addEventListener('click', func);
3
4 function func() {
5   console.log('!!!');
6   this.removeEventListener('click', func);
7 }
```

№1

⊗jsPmDmHU

Дана ссылка и кнопка. По нажатию на кнопку добавьте в конец текста ссылки содержимое ее атрибута `href` в круглых скобках. Сделайте так, чтобы это добавление происходило лишь по первому нажатию.

№2

⊗jsPmDmHU

Дана кнопка, значением которой служит число 1. Сделайте так, чтобы по клику на эту кнопку ее значение каждый раз увеличивалось на единицу. После того, как значение кнопки достигнет 10 - отвяжите обработчик события, чтобы кнопка больше не реагировала на нажатие.

Отвязывание обработчиков событий в цикле в JavaScript

Пусть теперь у нас есть не один элемент, а несколько. Например, несколько абзацев:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Давайте к каждому из этих абзацев обработчиком клика привяжем функцию `func`:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', func);
5 }
6
7 function func() {
8   console.log(this.textContent);
9 }
```


Давайте теперь переделаем код так, чтобы каждый абзац реагировал только на первое нажатие на него. Для этого при клике на абзац будем отвязывать от него привязанный обработчик. При этом отвязывание будет конкретно от этого абзаца, никак не затрагивая остальных.

Как вы уже знаете, элемент, в котором произошло событие, можно получить в функции-обработчике через `this`. Это значит, что нужно выполнять отвязывание обработчика от `this`, вот так:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', func);
5 }
6
7 function func() {
8   console.log(this.textContent);
9   this.removeEventListener('click', func); // отвязываем обработчик
10 }
```

№ 1

©jsPmDmLHU

Даны абзацы. По нажатию на любой из абзацев добавьте ему в конец восклицательный знак. Сделайте так, чтобы это добавление происходило лишь по первому нажатию.

Отвязывание анонимных функций в JavaScript

Пусть теперь к нашим абзацам привязана анонимная функция:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     console.log(this.textContent);
6   });
7 }
```

Пусть мы хотим отвязать эту функцию от абзаца после клика по этому абзацу. Нас, однако, ждет проблема: у функции нет имени, а значит мы не сможем обратиться к ней по этому имени, чтобы отвязать ее.

Для решения проблемы нужно дать имя функции, сделав из нее именованное функциональное выражение. Сделаем это:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() { // дали имя функции
5     console.log(this.textContent);
6   });
7 }
```

Теперь эту функцию можно отвязать внутри нее самой:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() {
5     console.log(this.textContent);
6     this.removeEventListener('click', func); // отвязываем функцию
7   });
8 }
```

№1

⊗ jsPmDmAHU

Дан список `ul`, в каждом пункте которого записано число. Сделайте так, чтобы по клику на любую `li` ее число увеличивалось на единицу.

№2

⊗ jsPmDmAHU

Модифицируйте предыдущую задачу так, чтобы каждая `li` увеличивала свое значение только по первому нажатию на нее.

№3

⊗ jsPmDmAHU

Модифицируйте предыдущую задачу так, чтобы каждая `li` увеличивала свое значение только если ее значение меньше 10.

Советы по написанию кода на примере DOM в JavaScript

Пусть перед вами стоит задача достаточной сложности, для реализации которой нужно написать некоторое количество строк кода.

Неправильным подходом будет пытаться написать весь код решения целиком, а потом начать его проверять. В этом случае высока вероятность, что у вас ничего не заработает, а ошибку придется искать в большом количестве кода.

Правильным подходом является разбиение задачи на маленькие элементарные шаги, которые вы будете реализовывать и сразу проверять их правильность. В этом случае, даже если вы где-то ошибетесь, вы сразу заметите проблему и исправите ее.

Давайте попробуем на практике. Пусть у вас есть абзацы:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4 <p>4</p>
5 <p>5</p>
6 <p>6</p>
```

Пусть перед вами стоит задача найти абзацы с числами, кратными 3, и найти сумму их чисел.

Первым маленьким шагом я бы предложил получить наши абзацы в виде массива и вывести этот массив в консоль, чтобы посмотреть, что мы все правильно получили. Сделаем это:

```
1 let elems = document.querySelectorAll('p');
2 console.log(elems);
```

Следующим маленьким шагом нужно перебрать наши абзацы циклом и вывести каждый из них в консоль по отдельности:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   console.log(elem);
5 }
```

А теперь в цикле выведем тексты наших абзацев:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   console.log(elem.textContent);
5 }
```

Теперь давайте выведем тексты тех абзацев, чье число делится на 3:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   let text = +elem.textContent;
5
6   if (text % 3 === 0) {
7     console.log(text);
8   }
9 }
```

Убедившись, что мы получаем правильные абзацы, можно приступить к суммированию их чисел:

```
1 let elems = document.querySelectorAll('p');
2 let sum = 0;
3
4 for (let elem of elems) {
5   let text = +elem.textContent;
6
7   if (text % 3 === 0) {
8     sum += text;
9   }
10 }
11
12 console.log(sum);
```

№ 1

⊗jsPmDmCdg

Дан список с годами:

```
1 <ul>
2   <li>2000</li>
3   <li>2004</li>
4   <li>2021</li>
5   <li>2022</li>
6   <li>2025</li>
7   <li>2031</li>
8 </ul>
```

Получите года, сумма цифр которых равна 6. Найдите сумму полученных годов.

Поиск ошибок в коде с DOM в JavaScript

В следующих задачах некоторый программист написал код и, возможно, допустил в нем ошибки. Вы должны проверить, делает ли код то, что описано. Если код работает некорректно, вы должны исправить ошибки.

№ 1

⊗jsPmDmMst

Код должен добавить текст в конец каждого абзаца:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelectorAll('p');
2 elems.textContent += '!';
```

№ 2

⊗jsPmDmMst

По клику на абзац его значение должно увеличиться на единицу:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     this.textContent += Number(1);
6   });
7 }
```

№3

⊗jsPmDmMst

По клику на кнопку текст абзаца должен стать жирным:

```
1 <p>text</p>
2 <button>click</button>

1 let button = document.querySelector('button');
2 let elem   = document.querySelector('p');
3
4 button.addEventListener('click', function() {
5   elem.innerHTML = '<b>elem.innerHTML</b>';
6 });
```

№4

⊗jsPmDmMst

По клику на кнопку должна вывестись сумма чисел из абзацев:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems  = document.querySelectorAll('p');
3
4 button.addEventListener('click', function() {
5   let sum = 0;
6
7   for (let elem of elems) {
8     sum += Number(elem.textContent);
9   }
10
11   console.log(sum);
12 });
```

№5

⊗jsPmDmMst

По клику на абзац ему в конец должен добавиться заданный текст:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', () => {
5     this.textContent += '!';
6   });
7 }
```

№6

⊗ jsPmDmMst

По клику на кнопку текст каждого абзаца должен стать жирным:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
4 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('p');
3
4 button.addEventListener('click', function() {
5   for (let elem of elems) {
6     elem.innerHTML = '<b>'+elem.innerHTML+'</b>';
7   }
8 });
```

№7

⊗ jsPmDmMst

По клику на кнопку в консоль должна выводиться сумма чисел из абзацев:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('p');
3 let sum = 0;
4
5 for (let elem of elems) {
6   sum = elem.textContent + 1;
7
8   button.addEventListener('click', function() {
9     console.log(sum);
10   });
11 }
```

№8

⊗ jsPmDmMst

В инпуты вводятся числа. По клику на кнопку в консоль должна выводиться сумма чисел из инпутов:

```
1 <input>
2 <input>
3 <input>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('input');
3 let sum = 0;
4
5 for (let elem of elems) {
6   sum += elem.value;
7 }
8
9 button.addEventListener('click', function() {
10   console.log(sum);
11 });
```

№9

⊗ jsPmDmMst

В первые два инпута вводятся числа. По клику на кнопку в третий инпут должна выводиться сумма этих чисел:

```
1 <input id="inp1">
2 <input id="inp2">
3 <input id="inp3">
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('#btn');
2 let inp1 = document.querySelector('#inp1');
3 let inp2 = document.querySelector('#inp2');
4 let inp3 = document.querySelector('#inp3');
5
6 btn.addEventListener('click', function() {
7   inp3.textContent = inp1.textContent + inp2.textContent;
8 });
```


№10

⊗jsPmDmMst

В первые два инпута вводятся числа. По клику на кнопку в абзац должна вывестись сумма этих чисел:

```
1 <input id="inp1">
2 <input id="inp2">
3 <p id="res"></p>
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('btn');
2 let res = document.querySelector('res');
3 let inp1 = document.querySelector('inp1');
4 let inp2 = document.querySelector('inp2');
5
6 button.addEventListener('click', function() {
7   res.value = inp1.value + inp2.value;
8 });
```

№11

⊗jsPmDmMst

По клику на кнопку нужно проверить, что текст в каждом инпуте совпадает с текстом его data атрибута:

```
1 <input data-text="text1">
2 <input data-text="text2">
3 <input data-text="text3">
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('input')
2 let button = document.querySelector('#button')
3
4 button.addEventListener('click', function() {
5   for (let input of inputs) {
6     if (input.value === input.dataset) {
7       input.classList.add('right')
8     } else {
9       input.classList.add('wrong')
10    }
11  }
12 });
```

№12

⊗jsPmDmMst

По клику на кнопку нужно проверить, что текст в каждом инпуте совпадает с текстом соответствующего элемента массива:

```
1 <input>
2 <input>
3 <input>
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('input')
2 let button = document.querySelector('#button')
3
4 let texts = [
5   'text1',
6   'text2',
7   'text3',
8 ];
9
10 button.addEventListener('click', function() {
11   for (let input of inputs) {
12     for (let text of texts) {
13       if (input.value === text) {
14         input.classList.add('right')
15       } else {
16         input.classList.add('wrong')
17       }
18     }
19   }
20 });
```

№13

⊗jsPmDmMst

В инпуты вводятся числа. По клику на кнопку в консоль должна выводиться сумма введенных чисел:

```
1 <input>
2 <input>
3 <input>
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('inputs');
2 let btn = document.querySelector('#btn');
3 let sum = 0;
4
5 btn.addEventListener('click', function() {
6   for (let input of inputs) {
7     sum += Number(input);
8   }
9
10  console.log(sum);
11 });
```

№14

⊗jsPmDmMst

В первые два инпута вводятся числа. По клику на кнопку в третий инпут должна вывестись сумма этих чисел:

```
1 <input id="inp1">
2 <input id="inp2">
3 <input id="inp3">
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('#btn');
2 let inp1 = document.querySelector('#inp1');
3 let inp2 = document.querySelector('#inp2');
4 let inp3 = document.querySelector('#inp3');
5
6 let sum = inp1.value + inp2.value;
7
8 btn.addEventListener('click', function() {
9   inp3.value = sum;
10 });
```

№15

⊗jsPmDmMst

В инпут вводится число. По потери фокуса в консоль должна вывестись сумма цифр этого числа:

```
1 <input id="inp">

1 let inp = document.querySelector('#inp').value;
2
3 inp.addEventListener('blur', function() {
4   let digits = +inp.split('').
5   let sum = 0;
6
7   for (let digit of digits) {
8     sum += digit;
9   }
10
11   console.log(sum);
12 });
```

Отработка изученного материала на работу с DOM

№1

⊗jsPmDmPm

Дан инпут и абзац. По потери фокуса в инпуте запишите значение инпута в конец текста абзаца.

№2

⊗jsPmDmPm

Дано несколько инпутов, абзац и кнопка. По нажатию на кнопку получите числа, стоящие в этих инпутах и запишите их сумму в абзац.

№3

⊗jsPmDmPm

Дан инпут. В него вводится число. По потери фокуса найдите сумму цифр этого числа.

№4

⊗jsPmDmPm

Дан инпут. В него вводятся числа через запятую. По потери фокуса найдите среднее арифметическое этих чисел (сумма делить на количество).

№5

⊗jsPmDmPm

Даны 4 инпута. В первый инпут вводится ФИО через пробел. По потери фокуса запишите фамилию, имя и отчество в остальные 3 инпута.

№6

⊗jsPmDmPm

Дан инпут. В него вводится ФИО через пробел. ФИО вводится с маленькой буквы. Сделайте так, чтобы по потери фокуса инпутом, введенные фамилия, имя и отчество автоматически стали записанными с большой буквы (в этом же инпуте).

№7

⊗jsPmDmPm

Дан инпут. В него вводится текст. По потери фокуса узнайте количество слов в этом тексте.

№8

⊗jsPmDmPm

Дан инпут. В него вводится дата в формате 31.12.2016. По потери фокуса в этом же инпуте поставьте эту дату в формате 2016-12-31.

№9

⊗jsPmDmPm

Дан инпут. В него вводится слово. По нажатию на кнопку проверьте то, что это слово читается с начала и с конца одинаково (например, мадам).

№10

⊗jsPmDmPm

Дан инпут. В него вводится число. Проверьте по потери фокуса, что это число содержит внутри себя цифру 3.

№11

⊗jsPmDmPm

Даны несколько абзацев и кнопка. По нажатию на кнопку запишите в конец каждого абзаца его порядковый номер.

№12

⊗jsPmDmPm

Даны ссылки. Добавьте в конец каждой ссылки ее href в круглых скобках.

№13

⊗jsPmDmPm

Даны ссылки. Если ссылка начинается с http://, то добавьте ей в конец стрелку → (делается так: →).

№14

⊗jsPmDmPm

Даны несколько абзацев с числами. По нажатию на любой абзац запишите в него квадрат числа, которое в нем находится.

№ 15

⊗jsPmDmPm

Дан инпут. В него вводится дата в формате 31.12.2016. По потери фокуса узнайте день недели (словом), который приходится на эту дату.

№ 16

⊗jsPmDmPm

Дан инпут. Реализуйте кнопки +1, -1, которые будут увеличивать или уменьшать на 1 значение инпута. Сделайте так, чтобы это значение не могло стать меньше нуля.

№ 17

⊗jsPmDmPm

Дан инпут. Даны абзацы. Пусть в этот инпут записывается суммарное количество нажатий по этим абзацам.

№ 18

⊗jsPmDmPm

На странице есть дивы. В них есть текст. Обойдите все дивы и обрежьте тот текст, который в них стоит так, чтобы он стал длиной 10 символов. И добавьте троеточие в конец обрезанного текста

№ 19

⊗jsPmDmPm

Дан инпут и кнопка. По нажатию на кнопку сгенерируйте случайную строку из 8-ми символов и запишите в инпут.

№ 20

⊗jsPmDmPm

Дан инпут и кнопка. Пользователь вводит в инпут какую-то строку. По нажатию на кнопку перемешайте введенные символы случайным образом и запишите ее обратно в инпут.

№21

⊗jsPmDmPm

Дан инпут, кнопка и абзац. В инпут вводится температура в градусах Фаренгейта. По клику на кнопку выведите в абзац температуру в градусах Цельсия.

№22

⊗jsPmDmPm

Дан инпут, кнопка и абзац. В инпут вводится число. По нажатию на кнопку выведите в абзац факториал этого числа.

№23

⊗jsPmDmPm

Даны 3 инпута, кнопка и абзац. В инпуты вводятся коэффициенты квадратного уравнения. По нажатию на кнопку найдите корни этого уравнения и выведите их в абзац.

